You `throw`; I'll `try` to `catch` it

A (low-level) journey through exception handling

Javier López-Gómez

`using std::cpp`, 2026-03-18

About me

Javier Lopez-Gomez

jalopezg-git https://jalopezg.dev/

javier.lopez.gomez AT proton.me

- 2017–2020: PhD in Computer Science and Technology (ARCOS-UC3M)
  - Prototype implementation of P0542R5 (old C++ contracts)
  - @ CERN (2019): `cling`: definition shadowing
- 2020–2023: Senior Applied Fellow (SoFTware for experiments group, CERN)
  - RNTuple columnar storage, `cling` maintenance...
- **2024–(today): Senior Compiler Engineer (Zimperium, Inc.)**
  - Binary-to-binary obfuscation solution
- **2025–(today): Voting member of WG21 (ISOCPP)**

Acknowledgements

- *Richard Dzenis (Zimperium)* — for introducing me in the amazing journey of exception handling. Thank you!

C++ exceptions trivia (1)

Q1: Myth or truth: exception handling is slow (Y / N / Neutral?)

C++ exceptions trivia (1)

Q1:

MAYBE 🤔 | Most of the time it's YES, but note that

- It's usually okay for exceptional control flow
- Some representations / implementations may yield faster dispatch

C++ exceptions trivia (2)

Q2: Exceptions can *propagate* through `-fno-exceptions` code (Y / N / Neutral?)

C++ exceptions trivia (2)

YES ✅

Q2:

- `.cfi_xxx` assembler directives are still generated → ability to unwind those frames
- Participate in **unwind** but NOT in **EH** (cannot throw or catch).

C++ exceptions trivia (3)

Q3: C code can be compiled with `-fexceptions` (Y / N / Neutral?)

C++ exceptions trivia (3)

Q3:

YES ✅

- `.cfi_xxx` assembler directives are still generated → ability to unwind those frames

C++ exceptions trivia (4)

Q4: Exceptions are appropriate for embedded systems (Y / N / Neutral?)

C++ exceptions trivia (4)

Q4:

MAYBE (depends) 🤔 | **Rationale:** EH is based on stack unwinding...

- Non-deterministic (unbound) stack unwinding[a]
- Additional bloat, if unwind / EH logic is statically linked

[a] There are, however, some proposals to address this.

Case study

```cpp
void may_throw(const std::vector<int> &v) {
  if (v.size() > 8)
    throw std::runtime_error("Hello, world!");
}

void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges
      ::to<std::vector>();
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}
```

```cpp
int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0
      x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

} catch (const std::runtime_error &e) {
  std::cerr << "main: Caught std::runtime_error
      !\n";
  return -1;
}
```

▷ See in Godbolt

Contents

1 Introduction

2 Itanium ABI (clang / gcc): What does it say about EH?

3 Itanium-compliant EH: LSB (Linux Standard Base)

4 Itanium ABI EH: Watching it work!

5 Wrapping up

Introduction

Exception Handling: definition (1)

Definition from Wikipedia, "Exception handling" article

> "Exception handling is the process of responding to the occurrence of exceptions —anomalous or exceptional conditions requiring special processing— during the execution of a program. In general, an exception breaks the normal flow of execution and executes a pre-registered exception handler."

- **In programming languages:** an exception object is prepared, which is then said to be `throw`n; control flow transfers to the inner-most `catch` block that can handle it

Exception Handling: definition (1)

Definition from Wikipedia, “Exception handling” article

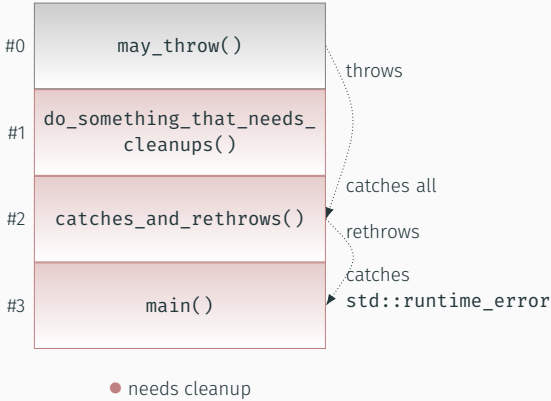
> “Exception handling is the process of responding to the occurrence of exceptions —anomalous or exceptional conditions requiring special processing— during the execution of a program. In general, an exception breaks the normal flow of execution and executes a pre-registered exceptiandler.”

- **In programming languages:** an exception object is prepared, which is then said to be `throw`n; control flow transfers to the inner-most `catch` block that can handle it

Exception Handling: definition (2)

Roughly, for C++...

1. **User code:** Allocate memory for exception of type T and throw it.
2. **Runtime:** Identify the proper catch block ($Catch_T$).
3. **Runtime:** Undo (unwind) affected stack frames; clean up as needed (dtors)
4. **Runtime:** Transfer control to $Catch_T$.



Exception Handling: definition (2)

Rough

1. U
   e
2. R
   b
3. R
   fr
4. R

- This is ABI-dependent!
- Runtime needs additional information emitted by the compiler!

l

ntime_error

needs cleanup

Let's talk about ABIs

Informal definition

Application Binary Interface (ABI): defines the low-level (machine-code-level) interfaces

ABI	API
low-level (binary) interface	high-level (source-code) interface
E.g., data layout, calling convention, ...	E.g., function name, parameter types, return type, ...

Well-known C++ ABIs:

- MSVC, used by Microsoft Visual C++
- Itanium C++ ABI, likely used by everything else (incl. `clang` / `gcc`)[1]

[1] `https://itanium-cxx-abi.github.io/cxx-abi/`

Let's talk about ABIs

Informal definition

Application Binary Interface (ABI): defines the low-level (machine-code-level) interfaces

AB

lo

E.g

re-

Wel

- MSVC, used by Microsoft Visual C++
- Itanium C++ ABI, likely used by everything else (incl. `clang` / `gcc`)[1]

A C++ ABI usually details

- Exception handling
- Layout of RTTI structures
- Function call 'lowering' (e.g. `this`) — usually no calling convention!
- Name mangling

[1] `https://itanium-cxx-abi.github.io/cxx-abi/`

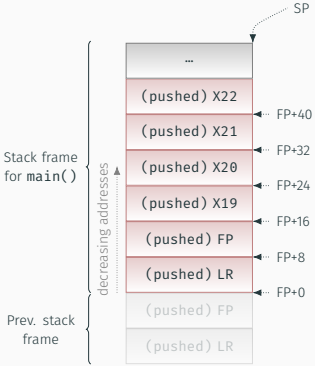
Let's talk about ABIs: Calling Conventions

A calling convention specifies...

- **Function arguments / return values:** register / mem?
- Which CPU registers are **scratch**, **caller-saved**, **callee-saved**?
- And, possibly, other stack frame layout details

May be specified by a complementary ABI:

- **x86_64, UNIX-based:** System V ABI[a].
- **x86_64, MSFT-based:** Microsoft x64 ABI
- **AArch64:** AAPCS64 (AArch64 Prodedure Call Standard)[b].



[a] https://refspecs.linuxbase.org/

[b] https://github.com/ARM-software/abi-aa/blob/main/aapcs64/aapcs64.rst

Itanium ABI (clang / gcc): What does it say about EH?

Itanium C++ ABI: EH

Itanium C++ ABI: Exception Handling

- Level I (Base ABI): language-independent (e.g. unwinder interface)
- Level II (C++ ABI): C++-specific (e.g. exception layout, allocation, ...)
- Level III (Suggested Implementation): some useful considerations; not seen here

Level I (Base ABI)

Definitions

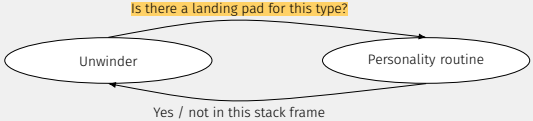
- **Stack unwind:** the (controlled) process of 'reverting' stack frames
  - Two-phase process: **search** and **cleanup**.
- At each step: call personality routine associated w/ each function in the call stack
  - **Personality routine:** interface between system unwinder library and language-specific EH semantics – guides unwind to a landing pad.

  - **Landing pad:** user-code that catches a given exception type OR performs clean-up
- Unwinder: runtime component able to perform stack unwind

Level I (Base ABI)

Definitions

- **Stack unwind:** the (controlled) process of 'reverting' stack frames
  - Two-phase process: **search** and **cleanup**.
- At each step: call personality routine associated w/ each function in the call stack
  - **Personality routine:** interface between system unwinder library and language-specific EH semantics – guides unwind to a landing pad.

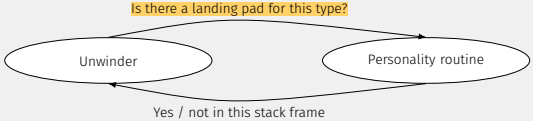


  - **Landing pad:** user-code that catches a given exception type OR performs clean-up
- Unwinder: runtime component able to perform stack unwind

Level I (Base ABI)

Definitions

- **Stack unwind:** the (controlled) process of 'reverting' stack frames
  - Two-phase process: **search** and **cleanup**.
- At each step: call personality routine associated w/ each function in the call stack
  - **Personality routine:** interface between system unwinder library and language-specific EH semantics – guides unwind to a landing pad.



  - **Landing pad:** user-code that catches a given exception type OR performs clean-up
- Unwinder: runtime component able to perform stack unwind

Level I (Base ABI)

Definitions

- **Stack unwind:** the (controlled) process of 'reverting' stack frames
- k
  c EH
  - `libgcc_s`: GCC-based toolchains
  - `libunwind`: provided by LLVM / compiler-rt; can be used by clang
- Unwinder: runtime component able to perform stack unwind

Level I (Base ABI): Raising an exception

- `_Unwind_RaiseException` : language-independent ABI function to raise an exception[2]

```
struct _Unwind_Exception;

_Unwind_Reason_Code _Unwind_RaiseException(struct _Unwind_Exception *
    exception_object);
```

[2] There is also `_Unwind_ForcedUnwind`, used for SJLJ / thread cancellation; not seen today!

Level I (Base ABI): Raising an exception

- on[2]

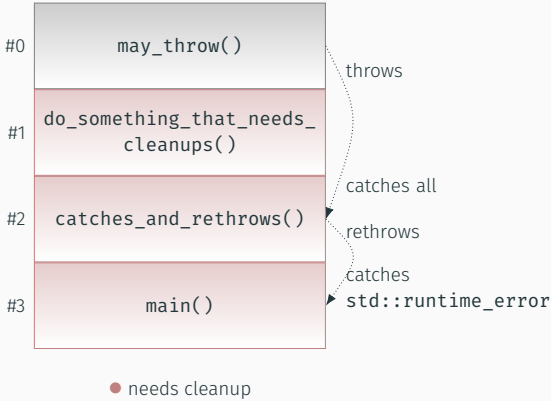
Language-independence! `_Unwind_Exception` is ∼ an exception 'header'...

```
struct _Unwind_Exception {
  uint64   exception_class; // 4 MSB = vendor
                            // 4 LSB = lang (C++ == "C++\0")
  _Unwind_Exception_Cleanup_Fn exception_cleanup;
  uint64   private_1;       // Unwinder-internal
  uint64   private_2;
};
```

[2] There is also `_Unwind_ForcedUnwind`, used for SJLJ / thread cancellation; not seen today!

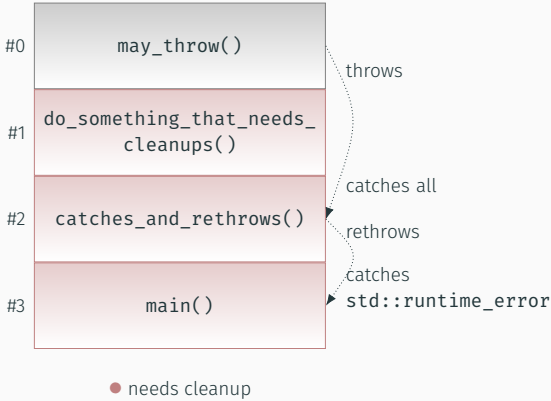
Level I (Base ABI): Phase 1 (Search)

- Find a stack frame that claims to catch the thrown type
  - Unwinder calls personality routine w/ `_UA_SEARCH_PHASE` at each step
  - Ignores cleanup landing pads
- **Stack in NOT unwound!**
- If found ($Catch_T$) → Phase 2; else Phase 1 returns `_URC_END_OF_STACK` → call `std::terminate()`



Level I (Base ABI): Phase 2 (Cleanup)

- Unwind one frame at a time, until reaching $Catch_T$'s frame
  - Personality routine is called w/ `_UA_CLEANUP_PHASE`
  - ...Stopping at cleanup landing pads!
- **Stack IS unwound!**
- Resume at $Catch_T$



Level I (Base ABI): Phase 2 (Cleanup)

- U
  re
- S
- R

Make it appear as if functions returned; continue at landing pad!

l

ntime_error

needs cleanup

Level II (C++ ABI)

Called from throwing code:

- `__cxa_allocate_exception(size_t thrown_size)`: allocates exception object of the given size
- `__cxa_throw(void *except, std::type_info *tinfo, void (*dtor)(void *))`: C++ ABI function for throwing an exception!

Called at landing pad:

- `__cxa_begin_catch(void *exceptionObject)`: book-keeping (increment refcount, etc.). Returns enclosing `__cxa_exception`.
- `__cxa_end_catch()`: book-keeping (decrement refcount; destroy if 0)
- `__cxa_rethrow()`: isn't it obvious?

Level II (C++ ABI)

Call

Call

`__cxa_exception` looks like...

```
struct __cxa_exception {
  std::type_info *   exceptionType;
  void               (*exceptionDestructor) (void *);
  unexpected_handler unexpectedHandler;  //  .- At throw point
  terminate_handler  terminateHandler;   // ,
  __cxa_exception *  nextException;

  int          handlerCount;
  int          handlerSwitchValue;   // .
  const char *actionRecord;          //  |
  const char *languageSpecificData;  //  |- Cached
  void *       catchTemp;            //  |
  void *       adjustedPtr;          // ,

  _Unwind_Exception unwindHeader;
};
```

Level II (C++ ABI)

Called from throwing code:

- `__cxa_allocate_exception(size_t thrown_size)`: allocates exception object of the

```
// +-------------------------+-----------------------------+---------------+
// | __cxa_exception         | _Unwind_Exception CLNGC++\0 | thrown object |
// +-------------------------+-----------------------------+---------------+
//                                                         ^
//                                                         |
```

▷ See `llvm-project:libcxxabi/src/cxa_exception.cpp`

Call

- `__cxa_rethrow()`: isn't it obvious?

Level II (C++ ABI)

Called from throwing code:

- `__cxa_allocate_exception(size_t thrown_size)`: allocates exception object of the

```
.LBB0_1:
        mov     edi, 16
        call    __cxa_allocate_exception@PLT
        mov     rbx, rax
        lea     rsi, [rip + .L.str]
        mov     rdi, rax
        call    std::runtime_error::runtime_error(char const*)@PLT
        mov     rsi, qword ptr [rip + typeinfo for std::runtime_error@GOTPCREL]
        mov     rdx, qword ptr [rip + std::runtime_error::~runtime_error()@GOTPCREL]
        mov     rdi, rbx
        call    __cxa_throw@PLT
```

▷ See in Godbolt

Call

- `__cxa_rethrow()`: isn't it obvious?

Level II (C++ ABI)

Call

Call

```
__cxa_throw(void *thrown_object, std::type_info *tinfo, void (_LIBCXXABI_DTOR_FUNC *dest)(void *)) {
#endif
  __cxa_eh_globals* globals = __cxa_get_globals();
  globals->uncaughtExceptions += 1; // Not atomically, since globals are thread-local

  __cxa_exception* exception_header = __cxa_init_primary_exception(thrown_object, tinfo, dest);
  exception_header->referenceCount = 1; // This is a newly allocated exception, no need for thread safety.

#if __has_feature(address_sanitizer)
  // Inform the ASan runtime that now might be a good time to clean stuff up.
  __asan_handle_no_return();
#endif

#ifdef __USING_SJLJ_EXCEPTIONS__
    _Unwind_SjLj_RaiseException(&exception_header->unwindHeader);
#else
    _Unwind_RaiseException(&exception_header->unwindHeader);
#endif
    //  This only happens when there is no handler, or some unexpected unwinding
    //     error happens.
    failed_throw(exception_header);
}
```

▷ See `llvm-project:libcxxabi/src/cxa_exception.cpp`

Level II (C++ ABI)

Level II also specifies...

- exception object layout
- common agreements for personality routine

BUT, it does NOT specify where/how unwind information is stored (!)

Itanium-compliant EH: LSB (Linux Standard Base)

Itanium-compliant EH: LSB (Linux Standard Base)

Linux Standard Base Core Specification 3.1[3]: 11.6. Exception Frames, says

- **Unwinder:** unwind tables (DWARF CFI-based)
- **Personality routine:** LSDA (Language-Specific Data Area)

[3] https://refspecs.linuxbase.org/LSB_3.1.1/LSB-Core-generic/LSB-Core-generic/ehframechpt.html

Itanium-compliant EH: LSB (Linux Standard Base)

Linux Standard Base Core Specification 3.1[3]: 11.6. Exception Frames, says

- **Unwinder:** unwind tables (DWARF CFI ed)
- **Personality routine:** LSDA (Language-Specific Data Area)

[3] https://refspecs.linuxbase.org/LSB_3.1.1/LSB-Core-generic/LSB-Core-generic/ehframechpt.html

Unwind tables: What?

Unwind tables

Conceptually: a HUGE table that specifies how to unwind any stack frame.

PC	LR (X30)	FP (X29)	X19	X20	X21	X22	X23	X24	…	D12	D13	D14
0x100000538	—	—	—	—	—	—	—	—	…	—	—	—
0x10000053C	SP+8	SP+0	—	—	—	—	—	—	…	—	—	—
0x100000540	SP+8	SP+0	SP+24	SP+16	—	—	—	—	…	—	—	—
0x100000544	SP+8 (FP+8)	SP+0 (FP+0)	SP+24 (FP+24)	SP+16 (FP+16)	—	—	—	—	…	—	—	—
0X100000548	SP+8 (FP+8)	SP+0 (FP+0)	SP+24 (FP+24)	SP+16 (FP+16)	—	—	—	—	…	—	—	—
0x10000054C	SP+8 (FP+8)	SP+0 (FP+0)	SP+24 (FP+24)	SP+16 (FP+16)	—	—	—	—	…	—	—	—
				…								
0x100000554	SP+8 (FP+8)	SP+0 (FP+0)	SP+24 (FP+24)	SP+16 (FP+16)	—	—	—	—	…	—	—	—
0x100000558	—	—	—	—	—	—	—	—	…	—	—	—
0x10000055C	—	—	—	—	—	—	—	—	…	—	—	—

(extract of) `_Z9may_throwRKSt6vectorIiSaIiEE`

Unwind tables

Conceptually: a HUGE table that specifies how to unwind any stack frame.

PC	LR (X30)	FP (X29)	X19	X20	X21							
0x100000538	—	—	—	—	—							
0x10000053C	SP+8	SP+0	—	—	—							
0x100000540	SP+8	SP+0	SP+24	SP+16	—							
0x100000544	SP+8 (FP+8)	SP+0 (FP+0)	SP+24 (FP+24)	SP+16 (FP+16)	—							
0X100000548	SP+8 (FP+8)	SP+0 (FP+0)	SP+24 (FP+24)	SP+16 (FP+16)	—							
0x10000054C	SP+8 (FP+8)	SP+0 (FP+0)	SP+24 (FP+24)	SP+16 (FP+16)	—							
				…								
0x100000554	SP+8 (FP+8)	SP+0 (FP+0)	SP+24 (FP+24)	SP+16 (FP+16)	—							
0x100000558	—	—	—	—	—	—	—	—	…	—	—	—
0x10000055C	—	—	—	—	—	—	—	—	…	—	—	—

(extract of) `_Z9may_throwRKSt6vectorIiSaIiEE`

```
may_throw(std::vector<int, std::allocator<int>> const&):
.Lfunc_begin0:
        .loc    1 8 0
        .cfi_startproc
        .cfi_personality 156, DW.ref.__gxx_personality_v0
        .cfi_lsda 28, .Lexception0
        stp     x29, x30, [sp, #-32]!
        .cfi_def_cfa_offset 32
        stp     x20, x19, [sp, #16]
        mov     x29, sp
        .cfi_def_cfa w29, 32
        .cfi_offset w19, -8
        .cfi_offset w20, -16
        .cfi_offset w30, -24
        .cfi_offset w29, -32
        .cfi_remember_state
.Ltmp29:
        .loc    3 1119 34 prologue_end
```

Unwind tables

Conceptually: a HUGE table that specifies how to unwind any stack frame.

0x1
0x1
0x1
0x1
0X1
0x1

0x1
0x1
0x1

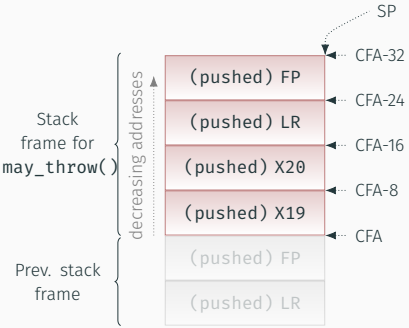
Space and time consuming! → Compressed form needed

(extract of) `_Z9MAY_THROWRKSt6vectorIiSaIiEE`

DWARF CFI (Call Frame Information): CFA

- DWARF[a], §6.4, specifies how to encode compressed form: Call Frame Information
- Stack frames are identified by their CFA (Canonical Frame Address)
  - Typically, **SP** at the call site in the prev. frame

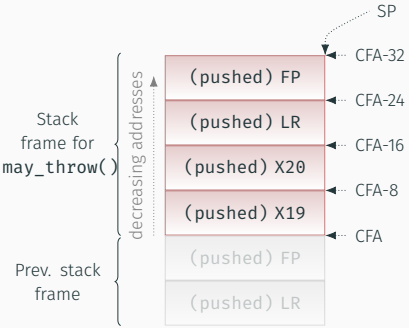
```
        .loc    1 16 3 is_stmt 1
        add     x0, sp, #24
        bl      may_throw(std::vector<int, std::allocator<int>> const&)
```



[a] https://dwarfstd.org/doc/DWARF5.pdf

DWARF CFI (Call Frame Information): CFA

```
may_throw(std::vector<int, std::allocator<int>> const&):
.Lfunc_begin0:
        .loc    1 8 0
        .cfi_startproc
        .cfi_personality 156, DW.ref.__gxx_personality_v0
        .cfi_lsda 28, .Lexception0
        stp     x29, x30, [sp, #-32]!
        .cfi_def_cfa_offset 32
        stp     x20, x19, [sp, #16]
        mov     x29, sp
        .cfi_def_cfa w29, 32
        .cfi_offset w19, -8
        .cfi_offset w20, -16
        .cfi_offset w30, -24
        .cfi_offset w29, -32
        .cfi_remember_state
.Ltmp29:
        .loc    3 1119 34 prologue_end
```



[1] https://dwarfstd.org/doc/DWARF5.pdf

DWARF CFI (Call Frame Information): CFI instructions (1)

```
may_throw(std::vector<int, std::allocator<int>> const&):
.Lfunc_begin0:
        .loc    1 8 0
        .cfi_startproc
        .cfi_personality 156, DW.ref.__gxx_personality_v0
        .cfi_lsda 28, .Lexception0
        stp     x29, x30, [sp, #-32]!
        .cfi_def_cfa_offset 32
        stp     x20, x19, [sp, #16]
        mov     x29, sp
        .cfi_def_cfa w29, 32
        .cfi_offset w19, -8
        .cfi_offset w20, -16
        .cfi_offset w30, -24
        .cfi_offset w29, -32
        .cfi_remember_state
.Ltmp29:
        .loc    3 1119 34 prologue_end
```

Usually, CFI instructions (opcodes) specify...

- An initial CFA rule (after prologue)
  → `DW_CFA_def_cfa_offset`
- CFA-relative position of callee-saved, FP, and return registers (LR)
  → `DW_CFA_offset`
- And... corrections of the CFA
  → `DW_CFA_def_cfa`
  → `DW_CFA_def_cfa_offset`
  → `DW_CFA_advance_loc`

DWARF CFI (Call Frame Information): CFI instructions (1)

Usually, CFI instructions (opcodes) specify...

- An initial CFA rule (after prologue)

```
may_throw(std::vector<int, std::allocator<int>> const&):
.Lfunc_
.Ltmp29
        .loc    3 1119 34 prologue_end
```

FP-based CFA rule is preferred!

ed,

→ `DW_CFA_advance_loc`

Unwind tables: Where?

.eh_frame and friends

Unwind information is serialized to some sections in the executable format, e.g.

```
$ llvm-objdump --section-headers ./case_study

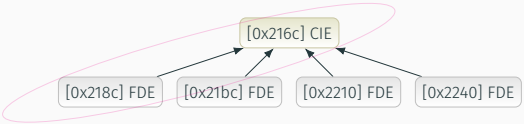
./case_study:  file format elf64-x86-64

Sections:
Idx Name                Size     VMA               Type
  0                     00000000 0000000000000000
…
 16 .eh_frame_hdr       00000044 00000000000020ec DATA
 17 .eh_frame           00000164 0000000000002130 DATA
 18 .gcc_except_table   0000007c 0000000000002294 DATA
…
```

.eh_frame: CIEs / FDEs

- A collection of “CFI” records:
  - CIE (Common Information Entry)
  - FDE (Frame Description Entry)

- Any number of FDEs can follow a CIE



.eh_frame: CIEs / FDEs

- A collection of "CFI" records:
  - CIE (Common Information Entry)
  - FDE (Frame Description Entry)
- Any number of FDEs can follow a CIE

CIE (Common Information Entry)

- Code and Data Alignment factors
- Return address (DWARF) register
- Augmentation string [+ data + length]
  - `z` : has data
  - `L` : has LSDA
  - `P` : has personality
- Initial CFI 'program'

`.eh_frame`: CIEs / FDEs

- A collection of “CFI” records:
  - CIE (Common Information Entry)
  - FDE (Frame Description Entry)
- Any number of FDEs can follow a CIE

FDE (Frame Description Entry)

- PC range
- Augmentation data
- CFI ‘program‘

.eh_frame: CIEs / FDEs

- A
- A

[0x216c] CIE version: 1

zPLR

align factor: C=1 D=-8

ret reg: 16

•P•

```
DW_CFA_def_cfa: reg7 +8
DW_CFA_offset: reg16 -8
```

[0x218c] FDE

PC range: [0x1220, +0x62]

L

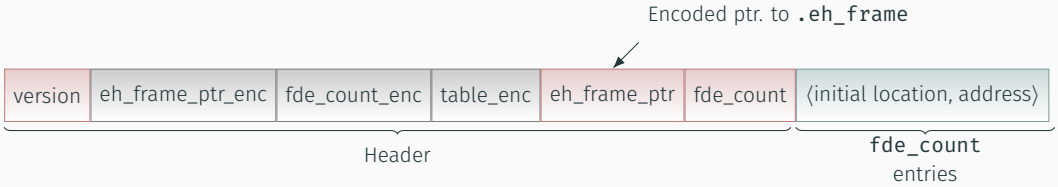
```
DW_CFA_advance_loc: 2
DW_CFA_def_cfa_offset: +16
DW_CFA_advance_loc: 1
DW_CFA_def_cfa_offset: +24
…
```

.eh_frame: Encoding

▷ See LSB_3.1.1/LSB-Core-generic/LSB-Core-generic/ehframechpt.html

`.eh_frame_hdr`: Accelerating lookup

`.eh_frame_hdr`: binary search table over FDEs in `.eh_frame`.



Compact Unwind: `__unwind_info` (macOS / iOS only)

Very efficient encoding of unwind information (given some assumptions)!

Good:

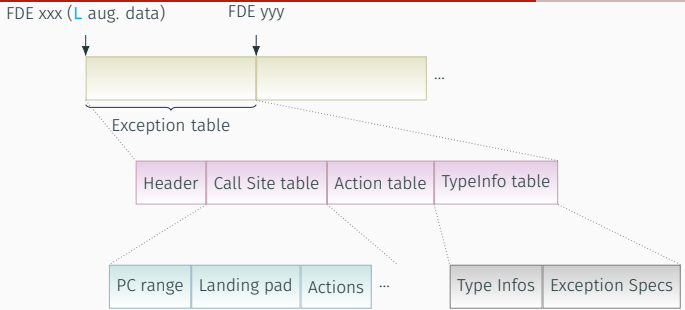
- Can encode a range's unwind-info in 32-bits (most of the time)
- Replaces both `.eh_frame` and `.eh_frame_hdr`

Bad:

- Only for Apple platforms

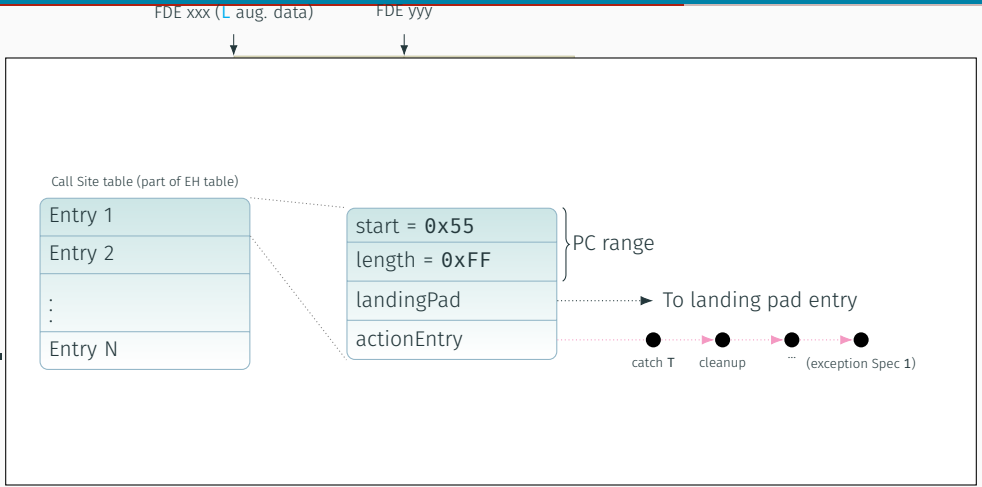
LSDA (for the `__gxx_personality_v0` personality routine)

LSDA: `.gcc_except_table`: Exception tables



- **Type Infos:** pointers to RTTI type information (`0` == `catch (...)`)
  - ```
    # >> Catch TypeInfos <<
    .long 0 # TypeInfo 2
    .Ltmp29: # TypeInfo 1
    .long .L_ZTISt13runtime_error.DW.stub-.Ltmp29
    ```
    → ```
    .L_ZTISt13runtime_error.DW.stub:
    .quad _ZTISt13runtime_error
    ```
- Actual encoding: ▷ See `llvm-project/…/libcxxabi/src/cxa_personality.cpp`

# LSDA: `.gcc_except_table`: Exception tables




- Actual encoding: ▷ See `llvm-project/…/libcxxabi/src/cxa_personality.cpp`

# C++ exceptions trivia (5)

Q5: Can I catch exceptions even if `-fno-rtti` is there (Y / N / Neutral?)

# C++ exceptions trivia (5)

MAYBE 🤔

**Q5:**

- Compiler generates minimal RTTI for the caught types
- But might be fragile[a]!

[a] https://github.com/llvm/llvm-project/issues/66117

# Itanium ABI EH: Watching it work!

# Itanium C++ EH: Recap

1. `__cxa_throw`: where throwing happens...
2. ...that calls `_Unwind_RaiseException`
3. ...that starts a two-phase (search + cleanup) stack unwind...
4. ...that, at each unwind step calls **personality** routine




5. ...until arriving at a **landing pad** that catches
   - while performing clean-up along the way...

# Itanium C++ EH: Recap

1. `__cxa_throw`: where throwing happens...
2. ...that calls `_Unwind_RaiseException`
3. ...that starts a two-phase (search + cleanup) stack unwind...
4. ...that, at each unwind step calls **personality** routine




5. ...until arriving at a **landing pad** that catches
   - while performing clean-up along the way...

# Itanium C++ EH: Recap

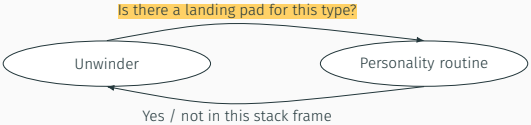
1. `__cxa_throw`: where throwing happens...
2. ...that calls `_Unwind_RaiseException`
3. ...that starts a two-phase (search + cleanup) stack unwind...
4. ...that, at each unwind step calls **personality** routine




5. ...until arriving at a **landing pad** that catches
   - while performing clean-up along the way...

# Itanium C++ EH: Recap

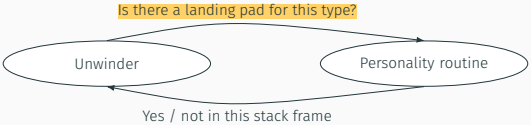
1. `__cxa_throw`: where throwing happens...
2. ...that calls `_Unwind_RaiseException`
3. ...that starts a two-phase (search + cleanup) stack unwind...
4. ...that, at each unwind step calls **personality** routine




5. ...until arriving at a **landing pad** that catches
   - while performing clean-up along the way...

# Itanium C++ EH: Recap

1. `__cxa_throw`: where throwing happens...
2. ...that calls `_Unwind_RaiseException`
3. ...that starts a two-phase (search + cleanup) stack unwind...
4. ...that, at each unwind step calls **personality** routine




5. ...until arriving at a **landing pad** that catches
   - while performing clean-up along the way...

# Itanium C++ EH: Recap

1. `__cxa_throw` : where throwing happens...
2.
3.
4. ...and where all the information is in
   - `.eh_frame` / `.eh_frame_hdr` (unwind info)
   - `.gcc_except_table` (LSDA)
5.
   - while performing clean-up along the way...

# Case study

```cpp
void may_throw(const std::vector<int> &v) {
  if (v.size() > 8)
    throw std::runtime_error("Hello, world!");
}

void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges
      ::to<std::vector>();
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}
```

```cpp
int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0
      x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

} catch (const std::runtime_error &e) {
  std::cerr << "main: Caught std::runtime_error
      !\n";
  return -1;
}
```

▷ See in Godbolt

Case study

- GNU/Linux (aarch64), clang 21.1.8, LLVM libunwind (Debug)

```
$ export LIBUNWIND_PRINT_APIS=1 LIBUNWIND_PRINT_UNWINDING=1 \
      LIBUNWIND_PRINT_DWARF=1 LIBCXXABI_PRINT_STATTAB=1
```

- `-fno-asynchronous-unwind-tables`

Getting to throw...

alarm@alarm:~

```
[alarm@alarm ~]$ tail -f case_study-stderr.txt
```

alarm@alarm:~

```
There is a running process, kill it and restart?: [Y/n] y
Process 1527 exited with status = 9 (0x00000009) killed
Process 1559 launched: '/home/alarm/case_study' (aarch64)
Process 1559 stopped
* thread #1, name = 'case_study', stop reason = breakpoint 6.1
    frame #0: 0x0000aaaaaaaaa1388 case_study`main + 36
case_study`main:
->  0xaaaaaaaaa1388 <+36>: bl     0xaaaaaaaaa1314 ; catches_and_rethrows(int)
    0xaaaaaaaaa138c <+40>: mov    w0, wzr
    0xaaaaaaaaa1390 <+44>: ldr    x19, [sp, #0x10]
    0xaaaaaaaaa1394 <+48>: ldp    x29, x30, [sp], #0x20
(lldb) register read x19 x20 x21 x22 fp lr
      x19 = 0x0000ffffffffffa58
      x20 = 0x0000aaaaaaaabfd58  case_study`
      x21 = 0x0000000000000002
      x22 = 0x0000ffffffffffa70
       fp = 0x0000ffffffffff8b0
       lr = 0x0000aaaaaaaaa1388  case_study`main + 36
(lldb)
case_study   breakpoint 6.1
```

case_study.cpp - GNU Emacs at alarm

File Edit Options Buffers Tools C++ Help

Save Undo

```
;
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

} catch (const std::runtime_error &e) {
  std::cerr << "main: Caught std::runtime_error!\n";
  return -1;
}
```

-:**- case_study.cpp Bot L34 (C++//l +2 Abbrev)

Mark set

Getting to throw...

alarm@alarm:~

```
[alarm@alarm ~]$ tail -f case_study-stderr.txt
```

alarm@alarm:~

```
    0xaaaaaaaaa1320 <+12>: bl     0xaaaaaaaaa1cc ; do_something_that_needs_clea
nups(int)
(lldb) c
Process 1559 resuming
Process 1559 stopped
* thread #1, name = 'case_study', stop reason = breakpoint 7.1
    frame #0: 0x0000aaaaaaaaa1320 case_study`catches_and_rethrows(int) + 12
case_study`catches_and_rethrows:
->  0xaaaaaaaaa1320 <+12>: bl     0xaaaaaaaaa1cc ; do_something_that_needs_clea
nups(int)
    0xaaaaaaaaa1324 <+16>: ldr    x19, [sp, #0x10]
    0xaaaaaaaaa1328 <+20>: ldp    x29, x30, [sp], #0x20
    0xaaaaaaaaa132c <+24>: ret
(lldb) register read x19 x20 x21 x22 fp lr
     x19 = 0x0000fffffffffa58
     x20 = 0x0000aaaaaaaabfd58  case_study`
     x21 = 0x0000000000000002
     x22 = 0x0000fffffffffa70
      fp = 0x0000fffffffff890
      lr = 0x0000aaaaaaaaa138c  case_study`main + 40
(lldb)
case_study   breakpoint 7.1
```

case_study.cpp - GNU Emacs at alarm

File Edit Options Buffers Tools C++ Help

Save Undo

```
;
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

} catch (const std::runtime_error &e) {
  std::cerr << "main: Caught std::runtime_error!\n";
  return -1;
}
```

-:**- case_study.cpp Bot L23 (C++//l +2 Abbrev)

Mark set

Getting to throw…

alarm@alarm:~

```
[alarm@alarm ~]$ tail -f case_study-stderr.txt
```

alarm@alarm:~

```
(lldb) c
Process 1559 resuming
Process 1559 stopped
* thread #1, name = 'case_study', stop reason = breakpoint 8.1
    frame #0: 0x0000aaaaaaaaa1298 case_study`do_something_that_needs_cleanups(i
nt) + 204
case_study`do_something_that_needs_cleanups:
->  0xaaaaaaaaa1298 <+204>: bl     0xaaaaaaaaa1160 ; may_throw(std::__1::vector<
int, std::__1::allocator<int>> const&)
    0xaaaaaaaaa129c <+208>: adrp   x8, 30
    0xaaaaaaaaa12a0 <+212>: ldr    x8, [x8, #0xfb0]
    0xaaaaaaaaa12a4 <+216>: ldr    x8, [x8]
(lldb) register read x19 x20 x21 x22 fp lr
     x19 = 0x0000aaaaaaac1230
     x20 = 0x000000000000000a
     x21 = 0x0000aaaaaaac1258
     x22 = 0x0000ffffffffffa70
      fp = 0x0000ffffffffff860
      lr = 0x0000aaaaaaaa120c  case_study`do_something_that_needs_cleanups(int
) + 64
(lldb)
case_study   breakpoint 8.1
```

case_study.cpp - GNU Emacs at alarm

File Edit Options Buffers Tools C++ Help

Save Undo

```
#include <vector>

void may_throw(const std::vector<int> &v) {
  if (v.size() > 8)
    throw std::runtime_error("Hello, world!");
}

void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges::to<std::vector>();
;
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;
```

-:**- case_study.cpp 11% L17 (C++//l +2 Abbrev)

Mark set

throw!

alarm@alarm:~

```
[alarm@alarm ~]$ tail -f case_study-stderr.txt
```

alarm@alarm:~

```
(lldb) register read x19 x20 x21 x22 fp lr
     x19 = 0x0000aaaaaaac1230
     x20 = 0x000000000000000a
     x21 = 0x0000aaaaaaac1258
     x22 = 0x0000ffffffffffa70
      fp = 0x0000ffffffffff860
      lr = 0x0000aaaaaaaa120c  case_study`do_something_that_needs_cleanups(int
) + 64
(lldb) c
Process 1559 resuming
Process 1559 stopped
* thread #1, name = 'case_study', stop reason = breakpoint 9.1
    frame #0: 0x0000aaaaaaaa11b4 case_study`may_throw(std::__1::vector<int, st
d::__1::allocator<int>> const&) + 84
case_study`may_throw:
-> 0xaaaaaaaaa11b4 <+84>: bl     0xaaaaaaaaa0f10 ; symbol stub for: __cxa_throw
   0xaaaaaaaaa11b8 <+88>: mov    x20, x0
   0xaaaaaaaaa11bc <+92>: mov    x0, x19
   0xaaaaaaaaa11c0 <+96>: bl     0xaaaaaaaaa0e70 ; symbol stub for: __cxa_free_
exception
(lldb)
case_study   breakpoint 9.1
```

case_study.cpp - GNU Emacs at alarm

File Edit Options Buffers Tools C++ Help

Save Undo

```
#include <vector>

void may_throw(const std::vector<int> &v) {
  if (v.size() > 8)
    throw std::runtime_error("Hello, world!");
}

void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges::to<std::vector>();
;
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

} catch (const std::runtime_error &e) {
```

-:**- case_study.cpp 11% L11 (C++//l +2 Abbrev)

Mark set

Phase 1: locating a landing pad

```
alarm@alarm:~
libunwind: __unw_get_reg(cursor=0xffffffffff550, regNum=-1, &value=0xffffffffff0a8)
libunwind: unwind_phase1(ex_obj=0xaaaaaaaa12c0): pc=0xaaaaaaaa11b8, start_ip=0xaaaaaaaa1160, func=.anonymous., lsda=0xaaaaaaaa187c, personality=0xffff7a55028
libunwind: unwind_phase1(ex_obj=0xaaaaaaaa12c0): calling personality function 0xffff7a55028
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffee60)
libunwind: _Unwind_GetLanguageSpecificData(context=0xffffffffff550) => 0xaaaaaaaa187c
libunwind: __unw_get_reg(cursor=0xffffffffff550, regNum=-1, &value=0xffffffffee98)
libunwind: _Unwind_GetIP(context=0xffffffffff550) => 0xaaaaaaaa11b8
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffee60)
libunwind: _Unwind_GetRegionStart(context=0xffffffffff550) => 0xaaaaaaaa1160
libunwind: unwind_phase1(ex_obj=0xaaaaaaaa12c0): _URC_CONTINUE_UNWIND
libunwind: __unw_step(cursor=0xffffffffff550)
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfc78)
DW_CFA_advance_loc: new offset=12
DW_CFA_def_cfa(reg=29, offset=32)
DW_CFA_offset(reg=19, offset=-8)
DW_CFA_offset(reg=20, offset=-16)
DW_CFA_offset(reg=30, offset=-24)
DW_CFA_offset(reg=29, offset=-32)
DW_CFA_nop
DW_CFA_nop
DW_CFA_nop
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfca0)
DW_CFA_advance_loc: new offset=20
DW_CFA_def_cfa(reg=29, offset=48)
DW_CFA_offset(reg=19, offset=-8)
DW_CFA_offset(reg=20, offset=-16)
DW_CFA_offset(reg=21, offset=-32)
DW_CFA_offset(reg=30, offset=-40)
DW_CFA_offset(reg=29, offset=-48)
DW_CFA_nop
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffff0c0)
libunwind: __unw_get_proc_name(cursor=0xffffffffff550, &buf=0xffffffffff108, bufLen=512)
libunwind: __unw_get_reg(cursor=0xffffffffff550, regNum=-1, &value=0xffffffffff0a8)
libunwind: unwind_phase1(ex_obj=0xaaaaaaaa12c0): pc=0xaaaaaaaaa129c, start_ip=0xaaaaaaaaa11cc, func=.anonymous., lsda=0xaaaaaaaa188c, personality=0xffff7a55028
libunwind: unwind_phase1(ex_obj=0xaaaaaaaa12c0): calling personality function 0xffff7a55028
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffee60)
libunwind: _Unwind_GetLanguageSpecificData(context=0xffffffffff550) => 0xaaaaaaaa188c
libunwind: __unw_get_reg(cursor=0xffffffffff550, regNum=-1, &value=0xffffffffee98)
libunwind: _Unwind_GetIP(context=0xffffffffff550) => 0xaaaaaaaaa129c
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffee60)
libunwind: _Unwind_GetRegionStart(context=0xffffffffff550) => 0xaaaaaaaaa11cc
libunwind: unwind_phase1(ex_obj=0xaaaaaaaa12c0): _URC_CONTINUE_UNWIND
libunwind: __unw_step(cursor=0xffffffffff550)
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfca0)
```

```
case_study.cpp  1% ln:  1  (F+//^0 +2 Abbrev)
Mark set
```

Phase 1: locating a landing pad

```
alarm@alarm:~
libunwind: _Unwind_GetRegionStart(context=0xffffffff550) => 0xaaaaaaaaa11cc
libunwind: unwind_phase1(ex_obj=0xaaaaaaaac12c0): _URC_CONTINUE_UNWIND
libunwind: __unw_step(cursor=0xffffffff550)
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfca0)
DW_CFA_advance_loc: new offset=20
DW_CFA_def_cfa(reg=29, offset=48)
DW_CFA_offset(reg=19, offset=-8)
DW_CFA_offset(reg=20, offset=-16)
DW_CFA_offset(reg=21, offset=-32)
DW_CFA_offset(reg=30, offset=-40)
DW_CFA_offset(reg=29, offset=-48)
DW_CFA_nop
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfcc4)
DW_CFA_advance_loc: new offset=12
DW_CFA_def_cfa(reg=29, offset=32)
DW_CFA_offset(reg=19, offset=-16)
DW_CFA_offset(reg=30, offset=-24)
DW_CFA_offset(reg=29, offset=-32)
DW_CFA_nop
libunwind: __unw_get_proc_info(cursor=0xffffffff550, &info=0xffffffff0c0)
libunwind: __unw_get_proc_name(cursor=0xffffffff550, &buffer=0xffffffff108, bufLen=512)
libunwind: __unw_get_reg(cursor=0xffffffff550, regNum=-1, &value=0xffffffff0a8)
libunwind: unwind_phase1(ex_obj=0xaaaaaaaac12c0): pc=0xaaaaaaaaa1324, start_ip=0xaaaaaaaaa1314, func=.anonymous., lsda=0xaaaaaaaaa18a0, personality=0xfffff7a55028
libunwind: unwind_phase1(ex_obj=0xaaaaaaaac12c0): calling personality function 0xfffff7a55028
libunwind: __unw_get_proc_info(cursor=0xffffffff550, &info=0xffffffffee60)
libunwind: _Unwind_GetLanguageSpecificData(context=0xffffffff550) => 0xaaaaaaaaa18a0
libunwind: __unw_get_reg(cursor=0xffffffff550, regNum=-1, &value=0xffffffffee98)
libunwind: _Unwind_GetIP(context=0xffffffff550) => 0xaaaaaaaaa1324
libunwind: __unw_get_proc_info(cursor=0xffffffff550, &info=0xffffffffee60)
libunwind: _Unwind_GetRegionStart(context=0xffffffff550) => 0xaaaaaaaaa1314
libunwind: __unw_get_reg(cursor=0xffffffff550, regNum=-2, &value=0xffffffff0a8)
libunwind: unwind_phase2(ex_obj=0xaaaaaaaac12c0): _URC_HANDLER_FOUND
libunwind: __unw_init_local(cursor=0xffffffff550, context=0xffffffff338)
parseFDEInstructions(instructions=0xfffff79d0ae4)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xfffff79d1cd4)
DW_CFA_advance_loc: new offset=4
DW_CFA_def_cfa_offset(16)
DW_CFA_offset(reg=29, offset=-16)
DW_CFA_offset(reg=30, offset=-8)
DW_CFA_advance_loc: new offset=12
DW_CFA_def_cfa_offset(1216)
DW_CFA_advance_loc: new offset=228
libunwind: unwind_phase2(ex_obj=0xaaaaaaaac12c0)
libunwind: __unw_step_stage2(cursor=0xffffffff550)
parseFDEInstructions(instructions=0xfffff79d0ae4)
```

```
File  Edit  Options  Buffers  Tools  C++  Help
Save  Undo
#include <vector>

void may_throw(const std::vector<int> &v) {
  if (v.size() > 8)
    throw std::runtime_error("Hello, world!");
}

void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges::to<std::vector>();
;
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

}
-:**-  case_study.cpp   11%  L11    (C++/l SP s2 abbrev)
Mark set
```

Phase 2: unwinding and resuming

```
alarm@alarm:~
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfca0)
DW_CFA_advance_loc: new offset=20
DW_CFA_def_cfa(reg=29, offset=48)
DW_CFA_offset(reg=19, offset=-8)
DW_CFA_offset(reg=20, offset=-16)
DW_CFA_offset(reg=21, offset=-32)
DW_CFA_offset(reg=30, offset=-40)
DW_CFA_offset(reg=29, offset=-48)
DW_CFA_nop
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfcc4)
DW_CFA_advance_loc: new offset=12
DW_CFA_def_cfa(reg=29, offset=32)
DW_CFA_offset(reg=19, offset=-16)
DW_CFA_offset(reg=30, offset=-24)
DW_CFA_offset(reg=29, offset=-32)
DW_CFA_nop
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffff0c0)
libunwind: __unw_get_proc_name(cursor=0xffffffffff550, &buf=0xffffffffff108, bufLen=512)
```

```
alarm@alarm:~
own_object, tinfo, dest);
(lldb) c
Process 1559 resuming
Process 1559 stopped
* thread #1, name = 'case_study', stop reason = breakpoint 10.1
    frame #0: 0x0000aaaaaaaaa12f8 case_study`do_something_that_needs_cleanups(int) + 300
case_study`do_something_that_needs_cleanups:
->  0xaaaaaaaaa12f8 <+300>: mov    x20, x0
    0xaaaaaaaaa12fc <+304>: cbz    x19, 0xaaaaaaaaa130c ; <+320>
    0xaaaaaaaaa1300 <+308>: sub    x1, x21, x19
    0xaaaaaaaaa1304 <+312>: mov    x0, x19
(lldb) register read x19 x20 x21 x22 fp lr
      x19 = 0x0000aaaaaaaac1230
      x20 = 0x000000000000000a
      x21 = 0x0000aaaaaaaac1258
      x22 = 0x0000ffffffffffa70
       fp = 0x0000ffffffffff860
       lr = 0x0000aaaaaaaaa12f8  case_study`do_something_that_needs_cleanups(int) + 300
(lldb)
case_study   breakpoint 10.1
```

```
case_study.cpp - GNU Emacs at alarm
File  Edit  Options  Buffers  Tools  C++  Help
Save  Undo
#include <vector>

void may_throw(const std::vector<int> &v) {
  if (v.size() > 8)
    throw std::runtime_error("Hello, world!");
}

void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges::to<std::vector>();
;
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

} catch (const std::runtime_error &e) {
-:**-  case_study.cpp   11%   L18   (C++//l +2 Abbrev)
Mark set
```

Phase 2: unwinding and resuming

```
alarm@alarm:~
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfca0)
DW_CFA_advance_loc: new offset=20
DW_CFA_def_cfa(reg=29, offset=48)
DW_CFA_offset(reg=19, offset=-8)
DW_CFA_offset(reg=20, offset=-16)
DW_CFA_offset(reg=21, offset=-32)
DW_CFA_offset(reg=30, offset=-40)
DW_CFA_offset(reg=29, offset=-48)
DW_CFA_nop
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfcc4)
DW_CFA_advance_loc: new offset=12
DW_CFA_def_cfa(reg=29, offset=32)
DW_CFA_offset(reg=19, offset=-16)
DW_CFA_offset(reg=30, offset=-24)
DW_CFA_offset(reg=29, offset=-32)
DW_CFA_nop
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffff6c0)
libunwind: __unw_get_proc_name(cursor=0xffffffffff550, &buf=0xffffffffff108, bufLen=512)
```

```
alarm@alarm:~
      lr = 0x0000aaaaaaaaa12f8  case_study`do_something_that_needs_cleanups(int
) + 300
(lldb) c
Process 1559 resuming
Process 1559 stopped
* thread #1, name = 'case_study', stop reason = breakpoint 11.1
    frame #0: 0x0000aaaaaaaaa1330 case_study`catches_and_rethrows(int) + 28
case_study`catches_and_rethrows:
-> 0xaaaaaaaaa1330 <+28>: bl     0xaaaaaaaaa0e80 ; symbol stub for: __cxa_begin
_catch
   0xaaaaaaaaa1334 <+32>: adrp   x0, 30
   0xaaaaaaaaa1338 <+36>: adrp   x1, 0
   0xaaaaaaaaa133c <+40>: add    x1, x1, #0x7be ;  + 46
(lldb) register read x19 x20 x21 x22 fp lr
     x19 = 0x0000fffffffffa58
     x20 = 0x0000aaaaaaabfd58  case_study`
     x21 = 0x0000000000000002
     x22 = 0x0000fffffffffa70
      fp = 0x0000fffffffff890
      lr = 0x0000aaaaaaaaa1330  case_study`catches_and_rethrows(int) + 28
(lldb)
case_study   breakpoint 11.1
```

```
case_study.cpp - GNU Emacs at alarm
File  Edit  Options  Buffers  Tools  C++  Help
Save  Undo
#include <vector>

void may_throw(const std::vector<int> &v) {
  if (v.size() > 8)
    throw std::runtime_error("Hello, world!");
}

void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges::to<std::vector>();
;
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

} catch (const std::runtime_error &e) {
-:**-  case_study.cpp   11%   L25    (C++/l +2 Abbrev)
Mark set
```

throw (rethrow)!

alarm@alarm:~

```
libunwind: __unw_set_reg(cursor=0xffffffff5b0, regNum=1, value=0x1)
libunwind: _Unwind_SetIP(context=0xffffffff5b0, value=0xaaaaaaaa1330)
libunwind: __unw_set_reg(cursor=0xffffffff5b0, regNum=-1, value=0xaaaaaaaa1330)
parseFDEInstructions(instructions=0xaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaabfcc4)
DW_CFA_advance_loc: new offset=12
DW_CFA_def_cfa(reg=29, offset=32)
DW_CFA_offset(reg=19, offset=-16)
DW_CFA_offset(reg=30, offset=-24)
DW_CFA_offset(reg=29, offset=-32)
DW_CFA_nop
libunwind: unwind_phase2(ex_obj=0xaaaaaaac12c0): _URC_INSTALL_CONTEXT
libunwind: __unw_get_reg(cursor=0xffffffff5b0, regNum=-1, &value=0xffffffffff108)
libunwind: __unw_get_reg(cursor=0xffffffff5b0, regNum=-2, &value=0xffffffffff100)
libunwind: unwind_phase2(ex_obj=0xaaaaaaac12c0): re-entering user code with ip=0xaaaaaaaa1330,
 sp=0xffffffffff890
libunwind: __unw_resume(cursor=0xffffffff5b0, walkedFrames=3)
libunwind: __unw_resume(cursor=0xffffffff5b0)
Catch all; rethrown
```

alarm@alarm:~

```
-> 454       _Unwind_Exception* unwind_exception = static_cast<_Unwind_Exceptio
n*>(unwind_arg);
   455       bool native_exception = __isOurExceptionClass(unwind_exception);
   456       __cxa_eh_globals* globals = __cxa_get_globals();
   457       // exception_header is a hackish offset from a foreign exception,
but it
(lldb) c
Process 1559 resuming
Process 1559 stopped
* thread #1, name = 'case_study', stop reason = breakpoint 15.1
    frame #0: 0x0000fffff7a53dfc libc++abi.so.1`__cxxabiv1::__cxa_rethrow() at
cxa_exception.cpp:628:50
   625       and that's ok.
   626   */
   627   void __cxa_rethrow() {
-> 628       __cxa_eh_globals* globals = __cxa_get_globals();
   629       __cxa_exception* exception_header = globals->caughtExceptions;
   630       if (NULL == exception_header)
   631           std::terminate();       // throw; called outside of a exception
 handler
(lldb)
case_study   cxa_exception.cpp:628:50   breakpoint 15.1
```

case_study.cpp - GNU Emacs at alarm

File Edit Options Buffers Tools C++ Help

Save Undo

```
#include <vector>

void may_throw(const std::vector<int> &v) {
  if (v.size() > 8)
    throw std::runtime_error("Hello, world!");
}

void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges::to<std::vector>();
;
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, int *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

} catch (const std::runtime_error &e) {
```

-:**- case_study.cpp 11% L27 (C++/l +2 Abbrev)

Mark set

Phase 1: locating a landing pad

```
alarm@alarm:~
libunwind: __unw_get_reg(cursor=0xffffffffff550, regNum=-1, &value=0xffffffffff0a8)
libunwind: unwind_phase1(ex_obj=0xaaaaaaaac12c0): pc=0xaaaaaaaa11b8, start_ip=0xaaaaaaaaa1160, func=.anonymous., lsda=0xaaaaaaaa187c, personality=0xffff7a55028
libunwind: unwind_phase1(ex_obj=0xaaaaaaaac12c0): calling personality function 0xffff7a55028
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffee60)
libunwind: _Unwind_GetLanguageSpecificData(context=0xffffffffff550) => 0xaaaaaaaaa187c
libunwind: __unw_get_reg(cursor=0xffffffffff550, regNum=-1, &value=0xffffffffee98)
libunwind: _Unwind_GetIP(context=0xffffffffff550) => 0xaaaaaaaaa11b8
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffee60)
libunwind: _Unwind_GetRegionStart(context=0xffffffffff550) => 0xaaaaaaaaa1160
libunwind: unwind_phase1(ex_obj=0xaaaaaaaac12c0): _URC_CONTINUE_UNWIND
libunwind: __unw_step(cursor=0xffffffffff550)
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfc78)
DW_CFA_advance_loc: new offset=12
DW_CFA_def_cfa(reg=29, offset=32)
DW_CFA_offset(reg=19, offset=-8)
DW_CFA_offset(reg=20, offset=-16)
DW_CFA_offset(reg=30, offset=-24)
DW_CFA_offset(reg=29, offset=-32)
DW_CFA_nop
DW_CFA_nop
DW_CFA_nop
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfca0)
DW_CFA_advance_loc: new offset=20
DW_CFA_def_cfa(reg=29, offset=48)
DW_CFA_offset(reg=19, offset=-8)
DW_CFA_offset(reg=20, offset=-16)
DW_CFA_offset(reg=21, offset=-32)
DW_CFA_offset(reg=30, offset=-40)
DW_CFA_offset(reg=29, offset=-48)
DW_CFA_nop
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffff0c0)
libunwind: __unw_get_proc_name(cursor=0xffffffffff550, &buf=0xffffffffff108, bufLen=512)
libunwind: __unw_get_reg(cursor=0xffffffffff550, regNum=-1, &value=0xffffffffff0a8)
libunwind: unwind_phase1(ex_obj=0xaaaaaaaac12c0): pc=0xaaaaaaaaa129c, start_ip=0xaaaaaaaaa11cc, func=.anonymous., lsda=0xaaaaaaaaa188c, personality=0xffff7a55028
libunwind: unwind_phase1(ex_obj=0xaaaaaaaac12c0): calling personality function 0xffff7a55028
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffee60)
libunwind: _Unwind_GetLanguageSpecificData(context=0xffffffffff550) => 0xaaaaaaaaa188c
libunwind: __unw_get_reg(cursor=0xffffffffff550, regNum=-1, &value=0xffffffffee98)
libunwind: _Unwind_GetIP(context=0xffffffffff550) => 0xaaaaaaaaa129c
libunwind: __unw_get_proc_info(cursor=0xffffffffff550, &info=0xffffffffee60)
libunwind: _Unwind_GetRegionStart(context=0xffffffffff550) => 0xaaaaaaaaa11cc
libunwind: unwind_phase1(ex_obj=0xaaaaaaaac12c0): _URC_CONTINUE_UNWIND
libunwind: __unw_step(cursor=0xffffffffff550)
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfca0)
```

```
#include <vector>

void may_throw(const std::vector<int> &v) {
  if (v.size() > 8)
    throw std::runtime_error("Bullies merely");
}

void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges::to<std::vector>();

  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

case_study.cpp  1:1  (C++//l +2 Abbrev)
Mark set
```

Phase 2: unwinding and resuming

```
alarm@alarm:~
libunwind: _Unwind_SetIP(context=0xffffffffd0, value=0xaaaaaaaa1350)
libunwind: __unw_set_reg(cursor=0xffffffffd0, regNum=-1, value=0xaaaaaaaa1350)
parseFDEInstructions(instructions=0xaaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaaabfcc4)
DW_CFA_advance_loc: new offset=12
DW_CFA_def_cfa(reg=29, offset=32)
DW_CFA_offset(reg=19, offset=-16)
DW_CFA_offset(reg=30, offset=-24)
DW_CFA_offset(reg=29, offset=-32)
DW_CFA_nop
libunwind: unwind_phase2(ex_obj=0xaaaaaaac12c0): _URC_INSTALL_CONTEXT
libunwind: __unw_get_reg(cursor=0xffffffffd0, regNum=-1, &value=0xffffffffff128)
libunwind: __unw_get_reg(cursor=0xffffffffd0, regNum=-2, &value=0xffffffffff120)
libunwind: unwind_phase2(ex_obj=0xaaaaaaac12c0): re-entering user code with ip=0xaaaaaaaa1350, sp=0xffffffffff800
libunwind: __unw_resume(cursor=0xffffffffd0, walkedFrames=3)
libunwind: __unw_resume(cursor=0xffffffffd0)
```

```
alarm@alarm:~
   625            and that's ok.
   626    */
   627    void __cxa_rethrow() {
-> 628        __cxa_eh_globals* globals = __cxa_get_globals();
   629        __cxa_exception* exception_header = globals->caughtExceptions;
   630        if (NULL == exception_header)
   631            std::terminate();        // throw; called outside of a exception
 handler
(lldb) c
Process 1559 resuming
Process 1559 stopped
* thread #1, name = 'case_study', stop reason = breakpoint 12.1
    frame #0: 0x0000aaaaaaaaa1350 case_study`catches_and_rethrows(int) + 60
case_study`catches_and_rethrows:
->  0xaaaaaaaaa1350 <+60>: mov    x19, x0
    0xaaaaaaaaa1354 <+64>: bl     0xaaaaaaaaa0f40 ; symbol stub for: __cxa_end_c
atch
    0xaaaaaaaaa1358 <+68>: mov    x0, x19
    0xaaaaaaaaa135c <+72>: bl     0xaaaaaaaaa0ef0 ; symbol stub for: _Unwind_Res
ume
(lldb)
case_study   breakpoint 12.1
```

```
case_study.cpp - GNU Emacs at alarm
File  Edit  Options  Buffers  Tools  C++  Help
Save  Undo
#include <vector>

void may_throw(const std::vector<int> &v) {
  if (v.size() > 8)
    throw std::runtime_error("Hello, world!");
}

void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges::to<std::vector>();
;
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

} catch (const std::runtime_error &e) {
-:**-  case_study.cpp   11%   L29    (C++//l +2 Abbrev)
Mark set
```

Phase 2: unwinding and resuming

```
alarm@alarm:~
libunwind: _Unwind_SetGR(context=0xffffffff600, reg=1, value=0x1)
libunwind: __unw_set_reg(cursor=0xffffffff600, regNum=1, value=0x1)
libunwind: _Unwind_SetIP(context=0xffffffff600, value=0xaaaaaaaaa13bc)
libunwind: __unw_set_reg(cursor=0xffffffff600, regNum=-1, value=0xaaaaaaaaa13bc)
parseFDEInstructions(instructions=0xaaaaaaabfc50)
DW_CFA_def_cfa(reg=31, offset=0)
parseFDEInstructions(instructions=0xaaaaaaabfce8)
DW_CFA_advance_loc: new offset=12
DW_CFA_def_cfa(reg=29, offset=32)
DW_CFA_offset(reg=19, offset=-16)
DW_CFA_offset(reg=30, offset=-24)
DW_CFA_offset(reg=29, offset=-32)
DW_CFA_nop
libunwind: unwind_phase2(ex_obj=0xaaaaaaac12c0): _URC_INSTALL_CONTEXT
libunwind: __unw_get_reg(cursor=0xffffffff600, regNum=-1, &value=0xffffffffff158)
libunwind: __unw_get_reg(cursor=0xffffffff600, regNum=-2, &value=0xffffffffff150)
libunwind: unwind_phase2(ex_obj=0xaaaaaaac12c0): re-entering user code with ip=0xaaaaaaaa13bc,
 sp=0xffffffff8b0
libunwind: __unw_resume(cursor=0xffffffff600, walkedFrames=3)
libunwind: __unw_resume(cursor=0xffffffff600)
```

```
alarm@alarm:~
    0xaaaaaaaaa135c <+72>: bl     0xaaaaaaaaa0ef0 ; symbol stub for: _Unwind_Res
ume
(lldb) c
Process 1559 resuming
Process 1559 stopped
* thread #1, name = 'case_study', stop reason = breakpoint 13.1
    frame #0: 0x0000aaaaaaaaa13c0 case_study`main + 92
case_study`main:
-> 0xaaaaaaaaa13c0 <+92>:  mov    x19, x0
    0xaaaaaaaaa13c4 <+96>:  b.ne   0xaaaaaaaaa13fc ; <+152>
    0xaaaaaaaaa13c8 <+100>: mov    x0, x19
    0xaaaaaaaaa13cc <+104>: bl     0xaaaaaaaaa0e80 ; symbol stub for: __cxa_begi
n_catch
(lldb) register read x19 x20 x21 x22 fp lr
      x19 = 0x0000ffffffffffa58
      x20 = 0x0000aaaaaaaabfd58  case_study`
      x21 = 0x0000000000000002
      x22 = 0x0000ffffffffffa70
       fp = 0x0000ffffffffff8b0
       lr = 0x0000aaaaaaaaa13bc  case_study`main + 88
(lldb)
case_study   breakpoint 13.1
```

```
case_study.cpp - GNU Emacs at alarm
File  Edit  Options  Buffers  Tools  C++  Help
Save  Undo
```

```
void do_something_that_needs_cleanups(int M) {
  // values are '{ 0, 1, ..., M-1 }'
  auto v = std::views::iota(0, M) | std::ranges::to<std::vector>();
;
  may_throw(v);
}

void catches_and_rethrows(int M) {
  try {
    std::string needs_cleanup{"str"};
    do_something_that_needs_cleanups(M);

  } catch (...) {
    std::cerr << "Catch all; rethrown\n";
    throw;
  }
}

int main(int argc, char *argv[]) try {
  assert(argc == 2);
  std::vector<unsigned char> vc{0x55, 0xaa, 0x55, 0xaa};
  catches_and_rethrows(atoi(argv[1]));
  return 0;

} catch (const std::runtime_error &e) {
  std::cerr << "main: Caught std::runtime_error!\n";
  return -1;
}
```

```
-:**-  case_study.cpp   Bot   L37   (C++//l +2 Abbrev)
Mark set
```

Wrapping up

A short note on MSVC / Windows EH

SEH

SEH (Structured Exception Handling): frame-based EH in Windows.

- x86_64 SEH: similar to what has been described here (`.pdata`, unwind ops)
- x86 SEH: different approach...
  - Handlers kept in a linked list pointed to by `fs` register
  - Function prologue (epilogue) registers (unregisters) handler

Ideas to take home

- Exception handling is a **complex** (but **amazing**!) topic 🙂
  - **Many ABI specs** (Itanium C++ ABI, architecture ABI, LSB, DWARF...)
- Per Itanium C++ ABI, unwinder and personality routine collaborate together 💃
- Unwind info / exception tables is stored in specific sections in the executable image
- EH is not generally fast, but mostly okay for *exceptional* control flow!

Other Resources (1/2)

1. Itanium C++ ABI: `https://itanium-cxx-abi.github.io/cxx-abi/abi-eh.html`
2. System V ABI Edition 4.1: `https://refspecs.linuxbase.org/elf/gabi41.pdf`
3. System V Application Binary Interface x86-64 Architecture Processor Supplement Draft Version: `https://refspecs.linuxbase.org/elf/x86_64-abi-0.99.pdf`
4. Procedure Call Standard for the Arm 64-bit Architecture (AArch64): `https://github.com/ARM-software/abi-aa/blob/main/aapcs64/aapcs64.rst`
5. Linux Standard Base (LSB): `https://refspecs.linuxbase.org/`
6. Linux Standard Base (LSB) Exception Frames: `https://refspecs.linuxbase.org/LSB_3.1.1/LSB-Core-generic/LSB-Core-generic/ehframechpt.html`
7. DWARF Version 5 Debugging Format Standard: `https://dwarfstd.org/doc/DWARF5.pdf`
8. The `.pdata` Section: `https://learn.microsoft.com/en-us/windows/win32/debug/pe-format#the-pdata-section`

Other Resources (2/2)

1. libcxxabi `__cxa_throw`: `https://github.com/llvm/llvm-project/blob/main/libcxxabi/src/cxa_exception.cpp#L281`
2. libcxxabi `__gxx_personality_v0`: `https://github.com/llvm/llvm-project/blob/main/libcxxabi/src/cxa_personality.cpp#L1025`
3. libunwind `_Unwind_RaiseException`: `https://github.com/llvm/llvm-project/blob/main/libunwind/src/UnwindLevel1.c#L466`
4. libunwind `UnwindCursor`: `https://github.com/llvm/llvm-project/blob/main/libunwind/src/UnwindCursor.hpp`

Thank you!

[Link to slides used in this presentation (CC BY-NC 4.0)]

Backup